

MAEG4060 Virtual Reality Systems and Applications

Course Project – Final Report

Project title: Interactive Physical Simulation of Off-Road Vehicle Dynamics in a Mountainous Terrain

Group members:

Guo Tsun Hei (1155186272): 3D Modeling, Texturing, Rigging, and Animation

Yim Fu Chong (1155176974): Programming, Physics Engine, and System Integration

Submission date: 11 April 2026

Submission deadline: 17 April 2026

Abstract

We present a DirectX 9.0c fixed-function **sandbox** (not a scored game) set on procedural mountainous terrain. The user drives a textured vehicle with keyboard or XInput gamepad; a third-person camera follows the chassis. The terrain uses bilinear height sampling, slope-based texture splatting (grass/rock), and surface normals for vehicle orientation. Static props include a crate (AABB) and a loose boulder (bounding sphere with gravity and impulses). A windmill model is loaded through the DirectX frame hierarchy with `ID3DXAnimationController` when present in the asset. A simple day/night cycle modulates the sun direction, ambient light, and clear colour; an `ID3DXFont` HUD shows speed and position. The implementation is native Visual C++17 with CMake, satisfying the course requirements for real-time 3D, input, physics-based motion, navigation on geometry, collision, and Maya-exported animation integration.

Contents

1	Introduction	3
1.1	Project scope	3
1.2	Alignment with course requirements	3
1.3	Mapping to marking criteria (group project)	3
2	Completed application	3
2.1	Feature overview	3
2.2	Executable and source layout	3
2.3	Screenshots	4
3	User guide	4
3.1	System requirements	4
3.2	Installation and running	4
3.3	Controls	4
3.3.1	Keyboard	4
3.3.2	Gamepad	4
3.4	Troubleshooting	4

4	Technical description	4
4.1	Architecture overview	7
4.2	DirectX 9 rendering	7
4.3	Terrain and model navigation	7
4.3.1	Bilinear interpolation for terrain height	7
4.3.2	Surface normal alignment	7
4.4	Animation	7
4.5	Collision detection and response	7
4.5.1	Axis-aligned bounding boxes (AABB)	7
4.5.2	Bounding spheres	7
4.5.3	Optional dynamic interaction	8
4.6	Physics and simulation	8
4.7	Environmental effects	8
4.8	Bonus work	8
5	3D models: sources and modifications	8
6	Conclusion	9
7	References	9
A	Appendix A: Build configuration	10
B	Appendix B: Third-party libraries	10
C	Appendix C: User Guide (standalone PDF)	11

1 Introduction

1.1 Project scope

The environment is an off-road hillside: the player explores a heightfield, interacts with props, and observes a time-varying sky palette. Compared to the Stage 1 proposal, terrain height data are **generated procedurally** at startup (Sketchfab terrain remained a visual reference) to keep the repository self-contained; splatting, vehicle dynamics, and collision behaviour match the intended design.

1.2 Alignment with course requirements

The application is implemented in **Visual C++** with **DirectX 9.0c**. It is a technical demonstration / sandbox (**not** a video game: no missions, no score). Input: **keyboard** and **gamepad**. The scene includes **physical simulation** (e.g. gravity, vehicle dynamics, terrain interaction, collisions).

1.3 Mapping to marking criteria (group project)

Table 1 lists Stage 3 items from the course project description; use the technical sections below to make each item easy for markers to find.

Table 1: Where to address each marked component (typical section pointers — adjust after writing).

Component (weight)	See section	Notes
Gamepad / joystick (5%)	§3.3	XInput / analog mapping
Model navigation (5%)	§4.3	Terrain following, camera or vehicle
Collision, static objects (10%)	§4.5	e.g. AABB / spheres
Physics-based motion (10%)	§4.6	Forces, integration, friction
Maya animation in VR (20%, group)	§4.4	.X / ID3DXAnimationController etc.
User guide (5%)	§3	Controls, requirements, run instructions
Report (15%)	Whole document	Clarity, completeness
Bonus (10%)	§4.8	Moving-object collision, extra effects, ...

2 Completed application

2.1 Feature overview

Procedural terrain mesh with bilinear sampling and two-pass rock splatting; simple longitudinal vehicle dynamics with steering, rolling resistance, and braking; kinematic terrain following (pose from surface normal); chase camera; crate and boulder models with AABB/sphere collision; boulder gravity and impulse from the vehicle hull; hierarchical windmill draw with animation time stepped each frame; day/night lighting and clear colour; on-screen HUD (ID3DXFont); resize-safe device reset path for the font.

2.2 Executable and source layout

Submit the Release binary as `Game_Build/maeg4060_stage2.exe` together with `Game_Build/Assets/` (see `scripts/package_game_build.bat` or CMake install). Source layout:

- Entry: `src/main.cpp`; simulation/render orchestration: `src/Game.cpp`.

- Rendering/device: `src/Graphics.cpp`; camera: `src/Camera.cpp`; static meshes: `src/ModelLoader.cpp`; hierarchy/animation: `src/AnimatedModel.cpp`; terrain: `src/Terrain.cpp`; vehicle: `src/Vehicle.cpp`; collision helpers: `src/Collision.cpp`; input: `src/Input.cpp`.
- Build: `CMakeLists.txt` (target `maeg4060_stage2`, links `d3d9`, `d3dx9`, `xinput9_1_0`).
- Assets: static props now support `Assets/models/replacements/*.obj + *.mtl + textures` with fallback to legacy `Assets/models/*.x`; terrain textures are generated at runtime if no image files are supplied.

2.3 Screenshots

Figures 1 and 2 show the running DirectX 9.0c sandbox and a GitHub Actions run page (workflow #14: boulder and windmill models) displaying the same application.

3 User guide

3.1 System requirements

Windows 10 or later (64-bit recommended), DirectX 9 compatible GPU, June 2010 DirectX SDK runtime components, display at least 1280×720 . Optional Xbox-compatible controller (XInput).

3.2 Installation and running

Run `maeg4060_stage2.exe` from `Game_Build` with `Assets` alongside it, or run from the repo root after CMake sets the VS debugger working directory. Build instructions are in the standalone user guide PDF.

3.3 Controls

3.3.1 Keyboard

Input	Action
W / S	Throttle forward / reverse
A / D	Steer left / right
Space	Brake
Esc	Exit application

3.3.2 Gamepad

Left stick: steering. Right trigger: forward throttle. Left trigger: brake and partial reverse (see `src/Input.cpp`).

3.4 Troubleshooting

Missing `d3dx9_*.dll`: install the legacy DirectX end-user runtime. Missing models: verify `Assets/models` relative to the process working directory. Black screen after resize: update GPU drivers; the sample calls `ID3DXFont::OnLostDevice/OnResetDevice` around `IDirect3DDevice9::Reset`.

4 Technical description

This section summarizes the main techniques used in the DirectX 9.0c (fixed-function pipeline) implementation: terrain sampling, vehicle-terrain coupling, collision choices, animation, lighting, and environmental effects. The application is a physics sandbox (not a scored game).

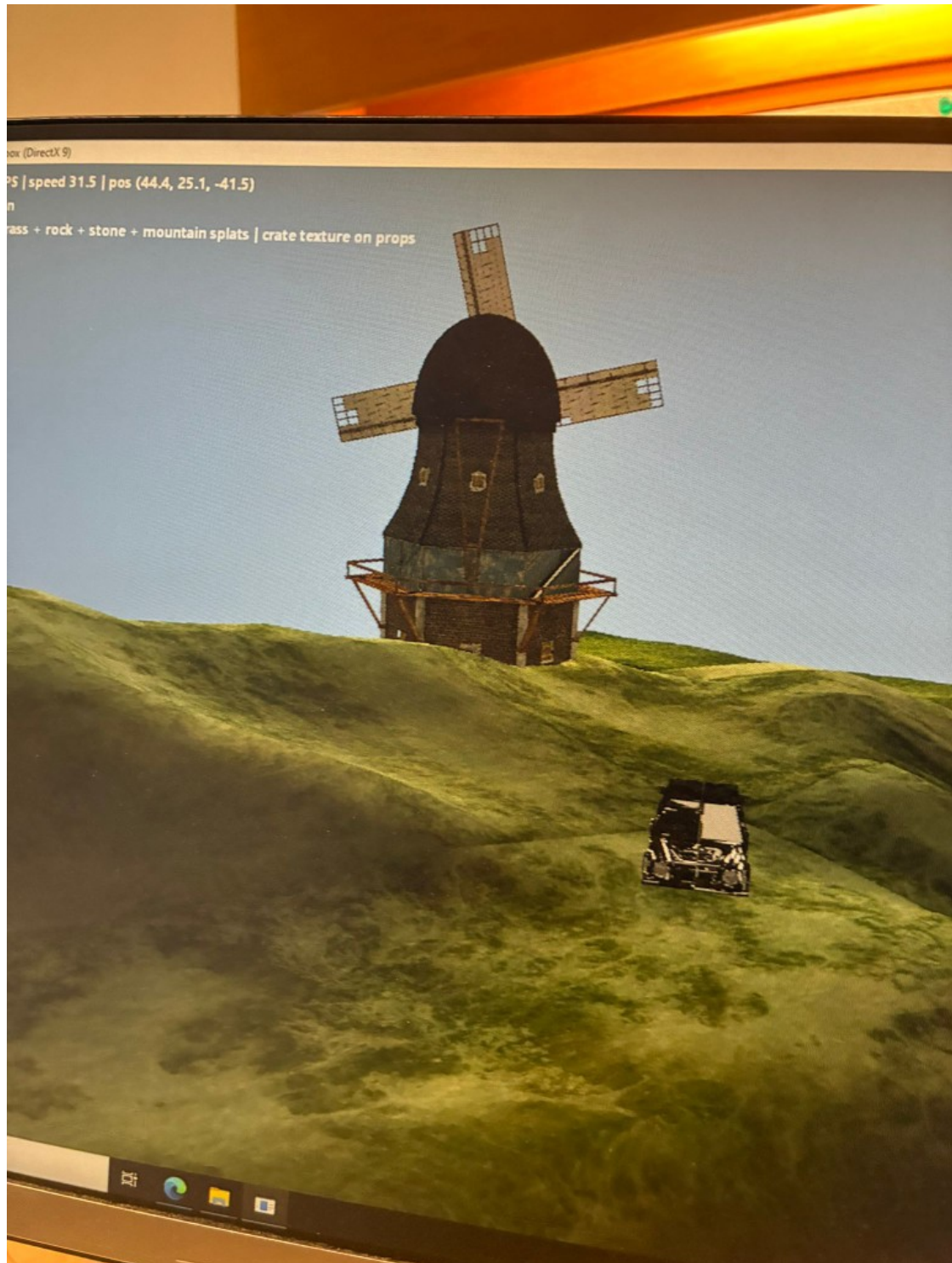


Figure 1: In-application view: procedural terrain with grass/rock/stone splats, windmill with hierarchy animation, props with crate texture, and HUD (speed, camera position, splat summary).

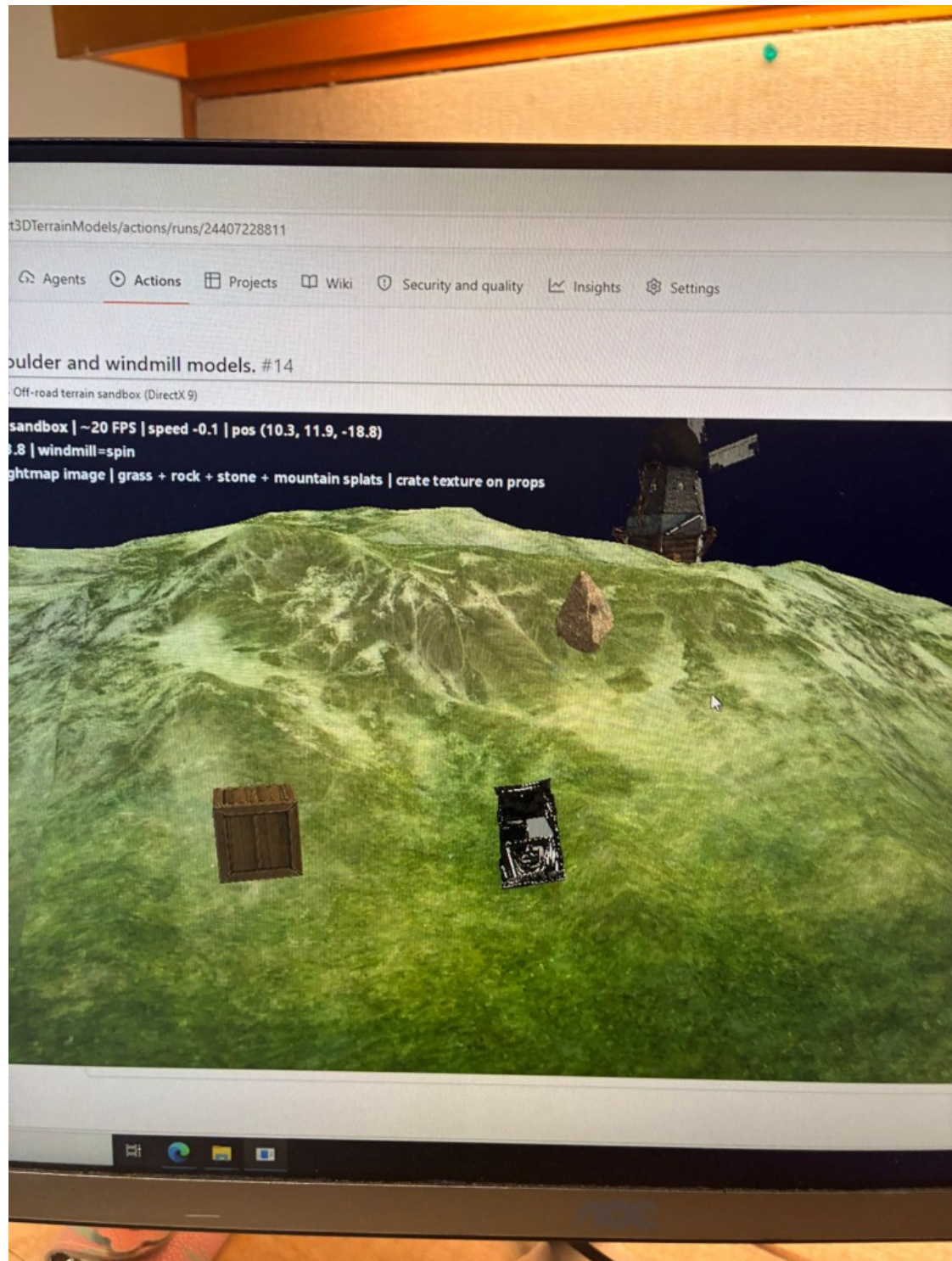


Figure 2: GitHub Actions: workflow run documenting boulder and windmill integration; inset shows the sandbox with windmill, boulder, crate, vehicle, and terrain HUD.

4.1 Architecture overview

The real-time loop follows input $\rightarrow$ simulation $\rightarrow$ rendering: keyboard via `GetAsyncKeyState`, gamepad via `XInputGetState`; `Game::Update` integrates the vehicle and boulder, resolves collisions, advances animation time, and updates lighting parameters; `Game::Render` sets the chase camera, draws terrain, hierarchy model, props, and vehicle with the fixed-function pipeline and a font overlay.

4.2 DirectX 9 rendering

The renderer uses a Direct3D 9 device with a depth-stencil buffer, textured meshes, and directional lighting. Meshes and animations are loaded from the `.X` format; the coordinate system is left-handed, consistent with `D3DXMatrixLookAtLH` and standard DirectX samples.

4.3 Terrain and model navigation

4.3.1 Bilinear interpolation for terrain height

The terrain height at an arbitrary world position (x, z) is obtained from a heightmap grid. To avoid discrete “stair-stepping” when the vehicle moves, the engine samples the four neighbouring heightmap texels and applies **bilinear interpolation**:

$$h(x, z) = (1 - t_x)(1 - t_z) h_{00} + t_x(1 - t_z) h_{10} + (1 - t_x)t_z h_{01} + t_x t_z h_{11},$$

where $t_x, t_z \in [0, 1]$ are fractional coordinates within the cell. This yields a continuous height field suitable for stable contact and visual alignment.

4.3.2 Surface normal alignment

The vehicle’s up vector is aligned with the **terrain surface normal**, computed from the local geometry (e.g. cross product of edges on the heightfield triangle under the vehicle). Pitch and roll follow the normal while yaw is driven by steering input, giving kinematic terrain-following without penetrating the mesh.

4.4 Animation

Maya keyframed animation for the windmill is exported in the `.X` format. The runtime uses `D3DXLoadMeshHierarchyFromX` with an `ID3DXAllocateHierarchy` implementation (`src/AnimatedModel.cpp`) and advances `ID3DXAnimationController` each frame. The jeep mesh is drawn as a single subset from `D3DXLoadMeshFromX` without per-wheel hierarchy in this build.

4.5 Collision detection and response

4.5.1 Axis-aligned bounding boxes (AABB)

Rectangular props (e.g. wooden crates) use **AABB** tests: fast intersection in world space along x , y , and z extents. Response can slide or push the vehicle along the minimum penetration axis.

4.5.2 Bounding spheres

Irregular objects (e.g. boulders) use **bounding sphere** collision: distance between sphere centres compared to the sum of radii. Spheres are cheap to update when objects rotate or roll and pair well with impulse-based response for loose rocks.

4.5.3 Optional dynamic interaction

Where implemented, **dynamic-to-dynamic** contact allows the vehicle to impart momentum to loose boulders so they roll on slopes and interact with other objects (see §4.8).

4.6 Physics and simulation

Gravity is applied to the boulder as a constant downward acceleration (engine constant ≈ 28 units/s² for stable stepping). The vehicle uses semi-implicit longitudinal integration with throttle, rolling resistance, and brake terms (`src/Vehicle.cpp`).

4.7 Environmental effects

A **day/night cycle** orbits the directional light and interpolates clear and ambient colours (`src/Game.cpp`). **Texture splatting** blends grass and rock in two passes using vertex alpha from slope. The HUD shows speed, world position, simulation time, and windmill load state.

4.8 Bonus work

Partial bonus: the boulder is dynamic (gravity, velocity, sphere versus vehicle AABB and crate AABB with positional correction and simple impulse when hit by the vehicle). Full rigid-body stacking and boulder–boulder coupling are not implemented.

5 3D models: sources and modifications

Per course requirements, external assets are listed with Sketchfab links and licenses (aligned with the Stage 1 proposal). **Modifications** should be updated to match what was actually done in Maya and the engine (retopo, materials, wheel hierarchy, scale, UVs, animation export, ...).

1. Half Dome Terrain Landscape

- Author: Studio Lab
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/half-dome-terrain-landscape-86ef075dc7d9414583ac2180e1e651a3>
- Usage: Reference for high-resolution terrain geometry and heightmap generation.
- Modifications: In-engine procedural heightfield (129×129 vertices, 200×200 world units) with analytic multi-frequency hills instead of the original Sketchfab mesh; runtime-generated grass/rock textures.

2. Controllable Vehicle (ATV)

- Title: ATV from game “PURE”
- Author: barking_dogo
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/atv-from-game-pure-f2e454611ebd4151b9d7aa31815c35e4>
- Usage: Main controllable vehicle model (wheel nodes separated for procedural animation).
- Modifications: Exported to `model_jeep.x`; scaled and placed with a world matrix from terrain normal and yaw; collision hull is a tuned AABB.

3. Rock Boulder (Game Ready Asset)

- Author: Aparicio Silva 3D
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/rock-boulder-game-ready-asset-0079cb0e717c4aeebe6bca21ec21b0d9>
- Usage: Dynamic physics / bounding-sphere collision.
- Modifications: Uniform scale to match gameplay radius ≈ 2.2 ; sphere collision and ground contact at terrain height plus radius.

4. Windmill

- Author: KIFIR
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/windmill-fe13862b9e3c4dc887d10e297d86f17f>
- Usage: Keyframed animation and environmental aesthetics.
- Modifications: Maya keyframes retained in `model_windmill.x`; playback via `ID3DXAnimationController` when the file contains animation sets.

5. Wooden Crate

- Author: Domingos Studios
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/wooden-crate-3c0259d17f3b4306890cdc809b545670>
- Usage: Static collision (AABB).
- Modifications: World placement at $(-35, h, 48)$ with AABB half-extents $(2.2, 2.2, 2.2)$ for collision.

6 Conclusion

The sandbox meets the core Stage 3 requirements in a compact codebase. Known limitations: no audio, no skinned character pipeline, jeep wheels are not independently articulated in code, and terrain is procedural rather than artist-authored DEM data. Future work could import a real heightmap PNG, add wheel bones from hierarchy, and extend physics to full rigid-body contact.

7 References

- Course project description (MAEG4060, 2025–26 Semester 2).
- DirectX SDK / MSDN documentation.
- External assets: cite consistently with §5.

A Appendix A: Build configuration

Visual Studio 2019 or later toolset recommended; CMake 3.20+; C++17; preprocessor `UNICODE/_UNICODE`; `DXSDK_DIR` for legacy headers/libs. Use Release builds for submission performance; `/W4` enabled on MSVC.

B Appendix B: Third-party libraries

If any (e.g. math helpers), list versions and licenses.

C Appendix C: User Guide (standalone PDF)

The following pages reproduce the submitted user guide (`User_Guide.pdf`): controls (keyboard and gamepad), hardware and software requirements, launch instructions, and troubleshooting.

MAEG4060 Course Project User Guide

Interactive Physical Simulation of Off-Road Vehicle Dynamics

Guo Tsun Hei; Yim Fu Chong

April 2026

1 How to start the application

- Open the `Game_Build` folder in your submission package.
- **Double-click** the main application executable (`.exe`) in that folder to launch the program.
- Keep the executable in the same folder structure as submitted so relative paths to assets (models, textures, heightmaps) resolve correctly. If the program reports missing files, run it from `Game_Build` without moving only the `.exe` away from its data folders.

2 Hardware and software requirements

- **Platform:** Windows PC (64-bit Windows recommended).
- **Graphics API:** The application uses **DirectX 9.0c**. You need a DirectX 9-compatible GPU and the **DirectX 9 End-User Runtime** (or equivalent system components) installed as required by your Windows version.
- **Display:** A desktop resolution of at least **1280×720** is recommended; other resolutions are supported when the window is resized.
- **Optional input:** An **XInput-compatible gamepad** (e.g. Xbox controller) for analog control. Keyboard control works without a gamepad.

3 Controls

3.1 Keyboard (WASD and related keys)

Input is handled via the keyboard using a typical first-person / vehicle-style layout (DirectInput), for example:

Input	Action
W / S	Forward / reverse (throttle)
A / D	Steer left / right
Space	Brake (or hand brake, depending on build)
Esc	Quit or return (if implemented)

Exact key bindings may match the on-screen HUD or readme in the binary package; the layout above reflects the project design (WASD movement with separate brake).

3.2 Gamepad (XInput)

With an XInput controller connected, the application maps analog axes and triggers for smoother control than the keyboard:

Control	Action
Left stick	Steering
Right trigger	Throttle (accelerate)
Left trigger	Brake / reverse (as implemented)
Start / Back	Pause or UI (if implemented)

Triggers provide analog throttle similar to a real vehicle; keyboard W/S act as digital throttle inputs.

4 Troubleshooting

- **Missing DLLs or DirectX errors:** Install or repair the DirectX 9 end-user runtime and ensure GPU drivers are up to date.
- **Black screen or failed device creation:** Try windowed mode, update graphics drivers, and confirm no other application has exclusive full-screen access to the GPU.
- **Assets not found:** Run from `Game_Build` with the supplied `Assets` (or equivalent) folder beside the executable.
- **Gamepad not detected:** Use a wired connection or official wireless adapter; confirm the controller appears in Windows as an XInput device.